Omar Ahmed

RSA

Tasks:[https://seedsecuritylabs.org/Labs_20.04/Files/Crypto_RSA/Crypto_RSA.pdf]

Task 1:

```c
int main ()
{
  BN_CTX *ctx = BN_CTX_new();

  BIGNUM *p = BN_new();
  BIGNUM *q = BN_new();
  BIGNUM *e = BN_new();
  BIGNUM *n = BN_new();
  BIGNUM *phiN = BN_new();
  BIGNUM *d = BN_new();

  BN_hex2bn(&p, "F7E75FDC469067FFDC4E847C51F452DF");
  BN_hex2bn(&q, "E85CED54AF57E53E092113E62F436F4F");
  BN_hex2bn(&e, "0D88C3");
  BN_mul(n, p, q, ctx); //n = p*q

  BIGNUM *temp1 = BN_new();
  BIGNUM *temp2 = BN_new();
  BIGNUM *one = BN_new();
  BN_dec2bn(&one, "1");
  BN_sub(temp1, p, one); //p-1
  BN_sub(temp2, q, one); //q-1
  BN_mul(phiN, temp1, temp2, ctx); //phiN = (p-1)(q-1)

  BN_mod_inverse(d, e, phiN, ctx); //d*emod phiN = 1
  printBN("p: ",p);
  printBN("q: ",q);
  printBN("n: ",n);
  printBN("phiN:",phiN);
  printBN("e: ",e);
  printBN("d: ",d);



  return 0;
}
```

More explanations can be found in the comments of the code above. The main part to understand is to derive d, we know that in RSA encryption e * d mod Φ(n) = 1, so we use BN_mod_inverse(d, e, phiN, ctx); //d*emod phiN = 1. The results of running the above code is:

```
[10/31/24]seed@VM:~/.../lab04$ gcc bn_sample.c -lcrypto
[10/31/24]seed@VM:~/.../lab04$ ./a.out
p:  F7E75FDC469067FFDC4E847C51F452DF
q:  E85CED54AF57E53E092113E62F436F4F
n:  E103ABD94892E3E74AFD724BF28E78366D9676BCCC70118BD0AA1968DBB143D1
phiN: E103ABD94892E3E74AFD724BF28E78348D52298BD687C44DEB3A81065A7981A
e:  0D88C3
d:  3587A24598E5F2A21DB007D89D18CC50ABA5075BA19A33890FE7C28A9B496AEB
```

Task 2:

```c
//TASK 2:
BN_CTX *ctx = BN_CTX_new();
BIGNUM *e = BN_new();
BIGNUM *n = BN_new();
// BIGNUM *phiN = BN_new();
BIGNUM *d = BN_new();
BIGNUM *M = BN_new(); //message
BIGNUM *C = BN_new(); //ciphertext
BIGNUM *V = BN_new(); //for verifying the encryption

//known values:
BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
BN_hex2bn(&e, "010001");
BN_hex2bn(&M, "4120746f702073656372657421"); //message, A top secret! converted to
BN_hex2bn(&d, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");

BN_mod_exp(C, M, e, n, ctx); //C = M^e mod n

BN_mod_exp(V, C, d, n, ctx); //V = C^d mod n

printBN("ciphertext: ",C);
printBN("decrypted cipher text: ",V);
printBN("original message: ",M);


return 0;
}
```

To encrypt using RSA, M^e mod n. To decrypt, use C^d mod n. (where M is the original message, and C is the cipher text). Below is the result of running the above code:

```
[11/02/24]seed@VM:~/.../lab04$ gcc bn_sample.c -lcrypto
[11/02/24]seed@VM:~/.../lab04$ ./a.out
ciphertext:  6FB078DA550B2650832661E14F4F8D2CFAEF475A0DF3A75CACDC5DE5CFC5FA
decrypted cipher text:  4120746F702073656372657421
original message:  4120746F702073656372657421
```

Task 3:

```c
//TASK 3:
BN_CTX *ctx = BN_CTX_new();
BIGNUM *e = BN_new();
BIGNUM *n = BN_new();
BIGNUM *d = BN_new();
BIGNUM *M = BN_new(); //message
BIGNUM *C = BN_new(); //ciphertext

//known values:
BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
BN_hex2bn(&e, "010001");
BN_hex2bn(&C, "8C0F971DF2F3672B28811407E2DABBE1DA0FEBBBDFC7DCB67396567EA1E2493F"); //given ciphert
BN_hex2bn(&d, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");

BN_mod_exp(M, C, d, n, ctx); //V = C^d mod n (decrypt)

printBN("ciphertext: ",C);
printBN("decrypted cipher text: ",M);


return 0;
}
```

In the same way as in task 2, we decrypt the ciphertext C using BN_mod_exp(M, C, d, n, ctx); //V = C^d mod n (decrypt). Below is the result of task 3, as well as converting the hex to text:

```
[11/02/24]seed@VM:~/.../lab04$ gcc bn_sample.c -lcrypto
[11/02/24]seed@VM:~/.../lab04$ ./a.out
ciphertext:  8C0F971DF2F3672B28811407E2DABBE1DA0FEBBBDFC7DCB67396567EA1E2493F
decrypted cipher text:  50617373776F72642069732064656573
[11/02/24]seed@VM:~/.../lab04$ python3  -c 'print(bytes.fromhex("50617373776F72642069732064656573").decode("utf-8
Password is dees
[11/02/24]seed@VM:~/.../lab04$
```

Task 4:

Converting original and modified message to hex:

```
[11/02/24]seed@VM:~/.../lab04$ python3 -c 'print("I owe you $2000.".encode("utf-8").hex(
49206f776520796f752024323030302e
[11/02/24]seed@VM:~/.../lab04$ python3 -c 'print("I owe you $3000.".encode("utf-8").hex(
49206f776520796f752024333030302e
```

```
//TASK 4:
BN_CTX *ctx = BN_CTX_new();
BIGNUM *e = BN_new();
BIGNUM *n = BN_new();
BIGNUM *d = BN_new();
BIGNUM *Moriginal = BN_new(); //original message
BIGNUM *Mmodified = BN_new(); //modified message
BIGNUM *Coriginal = BN_new(); //ciphertext for original message
BIGNUM *Cmodified = BN_new(); //ciphertext for modified message

//known values:
BN_hex2bn(&n, "DCBFFE3E51F62E09CE7032E2677A78946A849DC4CDDE3A4D0CB81629242FB1A5");
BN_hex2bn(&e, "010001");
BN_hex2bn(&Moriginal, "49206f776520796f752024323030302e"); //given message (I owe you $2000.) converted to he
BN_hex2bn(&Mmodified, "49206f776520796f752024333030302e"); //modified message (I owe you $3000.) converted to
BN_hex2bn(&d, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");

//use private key (d, n) for encryption:
BN_mod_exp(Coriginal, Moriginal, d, n, ctx); //Coriginal = Moriginal^e mod n (encrypt original message)
BN_mod_exp(Cmodified, Mmodified, d, n, ctx); //Cmodified = Mmodified^e mod n (encrypt modified message)

printBN("Signature of original message: ", Coriginal);
printBN("Signature of modified message: ", Cmodified);

return 0;
}
```

```
[11/02/24]seed@VM:~/.../lab04$ ./a.out
Signature of original message:  55A4E7F17F04CCFE2766E1EB32ADDBA890BBE92A6FBE2D785ED6E73CCB35E4
Signature of modified message:  BCC20FB7568E5D48E434C387C06A6025E90D29D848AF9C3EBAC0135D993058
```

The signatures are entirely different. The difference means that any modification of the original message will be able to be detected. Because there is a unique signature for every message, a sender cannot deny having signed a message. Overall, integrity, and non-repudiation are ensured.

Task 5:

Converting message to hex:

```
[11/02/24]seed@VM:~/.../lab04$ python3 -c 'print("Launch a missile.".encode("utf-8").hex(
4c61756e63682061206d697373696c652e
```

```c
//TASK 5:
BN_CTX *ctx = BN_CTX_new();
BIGNUM *e = BN_new();
BIGNUM *n = BN_new();
BIGNUM *d = BN_new();
BIGNUM *M = BN_new(); //message
BIGNUM *S = BN_new(); //signature
BIGNUM *Scorrupted = BN_new(); //corrupted signature
BIGNUM *Moriginal = BN_new(); //decrypted message from original signature
BIGNUM *Mcorrupted = BN_new(); //decrypted message from corrupted signature

//known values: (n changes from tasks 2-4), d is not provided
BN_hex2bn(&n, "AE1CD4DC432798D933779FBD46C6E1247F0CF1233595113AA51B450F18116115");
BN_hex2bn(&e, "010001");
BN_hex2bn(&M, "4c61756e63682061206d697373696c652e"); //given message (Launch a missile.) converted to hex
BN_hex2bn(&S, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6802F"); //Alice's signature
BN_hex2bn(&Scorrupted, "643D6F34902D9C7EC90CB0B2BCA36C47FA37165C0005CAB026C0542CBDB6803F"); //Alice's corrupted signa
// BN_hex2bn(&d, "74D806F9F3A62BAE331FFE3F0A68AFE35B3D2E4794148AACBC26AA381CD7D30D");

//decrypt message using public key (e, n)
BN_mod_exp(Moriginal, S, e, n, ctx); //Moriginal = S^e mod n

//decrypt corrupted signature
BN_mod_exp(Mcorrupted, Scorrupted, e, n, ctx); //Mcorrupted = Scorrupted^e mod n

printBN("Original message: ", M);
printBN("decrypted original signature: ", Moriginal);
printBN("decrypted corrupted signature: ", Mcorrupted);

return 0;
}
```

```
[11/02/24]seed@VM:~/.../lab04$ !gcc
gcc bn_sample.c -lcrypto
[11/02/24]seed@VM:~/.../lab04$ ./a.out
Original message:  4C61756E63682061206D697373696C652E
decrypted original signature:  4C61756E63682061206D697373696C652E
decrypted corrupted signature:  91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC
```

The signature is Alices, because when decrypting her signature with her public key (e, n), we get the original message. The decrypted corrupted signature is extremely different, indicating that the signature is not Alices. It also cannot be decoded in utf-8:

```
[11/02/24]seed@VM:~/.../lab04$ python3  -c 'print(bytes.fromhex("91471927C80DF1E42C154FB4638CE8BC726D3D66C83A4EB6B7BE0203B41AC294").
Traceback (most recent call last):
  File "<string>", line 1, in <module>
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x91 in position 0: invalid start byte
```

Task 6:

Fetch the certificates: openssl s_client -connect www.torontomu.ca:443 -showcerts

```
[11/03/24]seed@VM:~/.../Labsetup$ ls
bn_sample.c  Makefile  verify.c
[11/03/24]seed@VM:~/.../Labsetup$ openssl s_client -connect www.torontomu.ca:443 -showcerts
CONNECTED(00000003)
depth=2 C = US, ST = Texas, L = Houston, O = SSL Corporation, CN = SSL.com Root Certification Authority RSA
verify error:num=19:self signed certificate in certificate chain
verify return:1
depth=2 C = US, ST = Texas, L = Houston, O = SSL Corporation, CN = SSL.com Root Certification Authority RSA
verify return:1
depth=1 C = US, ST = Texas, L = Houston, O = SSL Corporation, CN = SSL.com RSA SSL subCA
verify return:1
depth=0 CN = torontomu.ca
verify return:1
---
Certificate chain
 0 s:CN = torontomu.ca
   i:C = US, ST = Texas, L = Houston, O = SSL Corporation, CN = SSL.com RSA SSL subCA
-----BEGIN CERTIFICATE-----
MIIG+jCCBOKgAwIBAgIQWGjkWgmDtm4GgWwcHQxAWjANBgkqhkiG9w0BAQsFADBp
MQswCQYDVQQGEwJVUzEOMAwGA1UECAwFVGV4YXMxEDAOBgNVBAcMB0hvdXN0b24x
GDAWBgNVBAoMD1NTTCBDb3Jwb3JhdGlvbjEeMBwGA1UEAwwVU1NMLmNvbSBSU0Eg
U1NMIHN1YkNBMB4XDTI0MDQxMTE0NTk0M1oXDTI1MDQxMzE0NTk0M1owFzEVMBMG
A1UEAwwMdG9yb250b211LmNhMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKC
AQEA8R7o+j+Nzb4NL0Zo7iGdDbkfpIBChPCOsJuDZyLhNYToZ1aY9jW6R94vjeqi
/kGF4Z7rC0byaEw2XvwSsymBEfopk5FLzYcbnchRvVd9JtaooPhW30goBL3dqOju
8j3LYH/qm0AH3Omw1g4hvllo//fL12XBZmaleQM5WT1fAa8xyA3nxeOPe+vYSFEX
exWbP+FdPgNw15zQBtOZPAFGeoQp+F+ENY+Y75AtanNqMIJaVQ3f2Rl4bfhs6ooy
nSYg/aC3hb99TinEzhxDDV3pEYx0T+XMnXLBkKK7ajy69v5dNL48X48acUEG08fc
KN9K+PRc2YaImkrlmyR/OC3t/QIDAQABo4IC7jCCAuowDAYDVR0TAQH/BAIwADAf
BgNVHSMEGDAWgBQmFH7g3Nem9+LUBCffYfHC7OcyyjByBggrBgEFBQcBAQRmMGQw
QAYIKwYBBQUHMAKGNGh0dHA6Ly9jZXJ0LnNzbC5jb20vU1NMY29tLVN1YkNBLVNT
TC1SU0EtNDA5Ni1SMS5jZXIwIAYIKwYBBQUHMAGGFGh0dHA6Ly9vY3Nwcy5zc2wu
Y29tMCkGA1UdEQQiMCCCDHRvcm9udG9tdS5jYYIQd3d3LnRvcm9udG9tdS5jYTAj
BgNVHSAEHDAaMAgGBmeBDAECATAOBgwrBgEEAYKpMAEDAQEwHQYDVR0lBBYwFAYI
KwYBBQUHAwIGCCsGAQUFBwMBMEUGA1UdHwQ+MDwwOqA4oDaGNGh0dHA6Ly9jcmxz
LnNzbC5jb20vU1NMY29tLVN1YkNBLVNTTC1SU0EtNDA5Ni1SMS5jcmwwDgYDVR0P
AQH/BAQDAgWgMIIBfQYKKwYBBAHWeQIEAgSCAW0EggFpAWcAdgBOdaMnXJoQwzhb
bNTfP1LrHfDgjhuNacCx+mSxYpo53wAAAY7NtSbNAAAEAwBHMEUCIQD51AFbILkr
rUbE4O6ue4CdUOI+b+EozKby05MBArcSmAIgd8Lc4OymPnZsYGKL4Kbm/YcDZxPd
we5P3pb7w6kwaSUAdgCi4wrkRe+9rZt+OO1HZ3dT14JbhJTXK14bLMS5UKRH5wAA
AY7NtSbYAAAEAwBHMEUCIQCSymXuKzuFn2V5S3Vc/KyjRGb2JG3KFI+J3t04km8B
hgIgfSFY1gjG7Z0qGVvhA9KsMPGGwdX00gu4frJAm8hXmVIAdQB9WR4S4Xgqexxh
Z3xe/fjQh1wUoE6VnrkDL9kOjC55uAAAAY7NtSbYAAAEAwBGMEQCIA1hWPXu9KFC
hCPzrXPt7iWPabWl0fGWz2l6o7tnF602AiAtc9bXn5Zpn+b3J6VVIEOUyi2svKrX
Fxa+Cls3Cp+VEDANBgkqhkiG9w0BAQsFAAOCAgEAKwBdRl95WATlfwFlLZkeCjor
AAL362kBt65bbk43jT+qMCe8x3ph0RxXDcTP8JXvhO/J8vzOXqftUHvsf0MIinri
Bz4NwLNeJ+Tq3MGjGYkeAV/E71eIrqDaR5s5K/lwSMaEa04nIJK5uMY+T2+Q2lEC
cX9iVt5Jox9kFTYYH+e683Qwhz7T9okq3MG1Q8j9cyD4eyzeJh32TDhFFl5l39qc
Qps3EinSUVn0U1BmaQ4PQyOk2QC/pXo5v0M9f36qa2Tb2Tcp/EdfLobgQsXZfaN7
8hY5HDDvfxfDj7+deL7YSB1Y35k9BIL0sOaAci9VbuJ3IyLfhA2kHNS9y7sjOlUx
QlzupaHCC380djAwOzH+RkDqp6W/WzzXYkXY8kb1iRo43A8G0SH3AfqKGLAjINdO
fttYbmbI17FJkScrU150FHWuZGs7AUrkTLDi6/3XBzzOYeB198QCNfQl9gt+QkHr
7CUo1oRi4n8Dh1v2+ixwMgOUOCiHuAR9TjCbEMwjK4RJNchNgeeBxiTLutaKlviJ
5zrH5rW1LvVktX53YclGqCSXeQ55Hi8impOt+Oq4y1WwTJHAaI4sSghyeUIz9r5W
+MtApDExgIyrTpTYrX6CZ6LBRb0I+SdXzkVj9Ox7GhSxn28td6OXTUxQxx87odqs
CXF533clMY/Aa0lwxBU=
-----END CERTIFICATE-----
 1 s:C = US, ST = Texas, L = Houston, O = SSL Corporation, CN = SSL.com RSA SSL subCA
   i:C = US, ST = Texas, L = Houston, O = SSL Corporation, CN = SSL.com Root Certification Authority RSA
-----BEGIN CERTIFICATE-----
MIIGbzCCBFegAwIBAgIICZftEJ0fB/wwDQYJKoZIhvcNAQELBQAwfDELMAkGA1UE
BhMCVVMxDjAMBgNVBAgMBVRleGFzMRAwDgYDVQQHDAdIb3VzdG9uMRgwFgYDVQQK
```

Save the server certificate (certificate 0) as c0.pem:

```
-----BEGIN CERTIFICATE-----
MIIG+jCCBOKgAwIBAgIQWGjkWgmDtm4GgWwcHQxAWjANBgkqhkiG9w0BAQsFADBp
MQswCQYDVQQGEwJVUzEOMAwGA1UECAwFVGV4YXMxEDAOBgNVBAcMB0hvdXN0b24x
GDAWBgNVBAoMD1NTTCBDb3Jwb3JhdGlvbjEeMBwGA1UEAwwVU1NMLmNvbSBSU0Eg
U1NMIHN1YkNBMB4XDTI0MDQxMTE0NTk0M1oXDTI1MDQxMzE0NTk0M1owFzEVMBMG
A1UEAwwMdG9yb250b211LmNhMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKC
AQEA8R7o+j+Nzb4NL0Zo7iGdDbkfpIBChPCOsJuDZyLhNYToZ1aY9jW6R94vjeqi
/kGF4Z7rC0byaEw2XvwSsymBEfopk5FLzYcbnchRvVd9JtaooPhW30goBL3dqOju
8j3LYH/qm0AH3Omw1g4hvllo//fL12XBZmaleQM5WT1fAa8xyA3nxeOPe+vYSFEX
exWbP+FdPgNw15zQBtOZPAFGeoQp+F+ENY+Y75AtanNqMIJaVQ3f2Rl4bfhs6ooy
nSYg/aC3hb99TinEzhxDDV3pEYx0T+XMnXLBkKK7ajy69v5dNL48X48acUEG08fc
KN9K+PRc2YaImkrlmyR/OC3t/QIDAQABo4IC7jCCAuowDAYDVR0TAQH/BAIwADAf
BgNVHSMEGDAWgBQmFH7g3Nem9+LUBCffYfHC7OcyyjByBggrBgEFBQcBAQRmMGQw
QAYIKwYBBQUHMAKGNGh0dHA6Ly9jZXJ0LnNzbC5jb20vU1NMY29tLVN1YkNBLVNT
TC1SU0EtNDA5Ni1SMS5jZXIwIAYIKwYBBQUHMAGGFGh0dHA6Ly9vY3Nwcy5zc2wu
Y29tMCkGA1UdEQQiMCCCDHRvcm9udG9tdS5jYYIQd3d3LnRvcm9udG9tdS5jYTAj
BgNVHSAEHDAaMAgGBmeBDAECATAOBgwrBgEEAYKpMAEDAQEwHQYDVR0lBBYwFAYI
KwYBBQUHAwIGCCsGAQUFBwMBMEUGA1UdHwQ+MDwwOqA4oDaGNGh0dHA6Ly9jcmxz
LnNzbC5jb20vU1NMY29tLVN1YkNBLVNTTC1SU0EtNDA5Ni1SMS5jcmwwDgYDVR0P
AQH/BAQDAgWgMIIBfQYKKwYBBAHWeQIEAgSCAW0EggFpAWcAdgBOdaMnXJoQwzhb
bNTfP1LrHfDgjhuNacCx+mSxYpo53wAAAY7NtSbNAAAEAwBHMEUCIQD51AFbILkr
rUbE4O6ue4CdUOI+b+EozKby05MBArcSmAIgd8Lc4OymPnZsYGKL4Kbm/YcDZxPd
we5P3pb7w6kwaSUAdgCi4wrkRe+9rZt+OO1HZ3dT14JbhJTXK14bLMS5UKRH5wAA
AY7NtSbYAAAEAwBHMEUCIQCSymXuKzuFn2V5S3Vc/KyjRGb2JG3KFI+J3t04km8B
hgIgfSFY1gjG7Z0qGVvhA9KsMPGGwdX00gu4frJAm8hXmVIAdQB9WR4S4Xgqexxh
Z3xe/fjQh1wUoE6VnrkDL9kOjC55uAAAAY7NtSbYAAAEAwBGMEQCIA1hWPXu9KFC
hCPzrXPt7iWPabWl0fGWz2l6o7tnF602AiAtc9bXn5Zpn+b3J6VVIEOUyi2svKrX
Fxa+Cls3Cp+VEDANBgkqhkiG9w0BAQsFAAOCAgEAKwBdRl95WATlfwFlLZkeCjor
AAL362kBt65bbk43jT+qMCe8x3ph0RxXDcTP8JXvhO/J8vzOXqftUHvsf0MIinri
Bz4NwLNeJ+Tq3MGjGYkeAV/E71eIrqDaR5s5K/lwSMaEa04nIJK5uMY+T2+Q2lEC
cX9iVt5Jox9kFTYYH+e683Qwhz7T9okq3MG1Q8j9cyD4eyzeJh32TDhFFl5l39qc
Qps3EinSUVn0U1BmaQ4PQyOk2QC/pXo5v0M9f36qa2Tb2Tcp/EdfLobgQsXZfaN7
8hY5HDDvfxfDj7+deL7YSB1Y35k9BIL0sOaAci9VbuJ3IyLfhA2kHNS9y7sjOlUx
QlzupaHCC380djAwOzH+RkDqp6W/WzzXYkXY8kb1iRo43A8G0SH3AfqKGLAjINdO
fttYbmbI17FJkScrU150FHWuZGs7AUrkTLDi6/3XBzzOYeB198QCNfQl9gt+QkHr
7CUo1oRi4n8Dh1v2+ixwMgOUOCiHuAR9TjCbEMwjK4RJNchNgeeBxiTLutaKlviJ
5zrH5rW1LvVktX53YclGqCSXeQ55Hi8impOt+Oq4y1WwTJHAaI4sSghyeUIz9r5W
+MtApDExgIyrTpTYrX6CZ6LBRb0I+SdXzkVj9Ox7GhSxn28td6OXTUxQxx87odqs
CXF533clMY/Aa0lwxBU=
-----END CERTIFICATE-----
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
"c0.pem" 40L, 2480C                                                    40,25         A
```

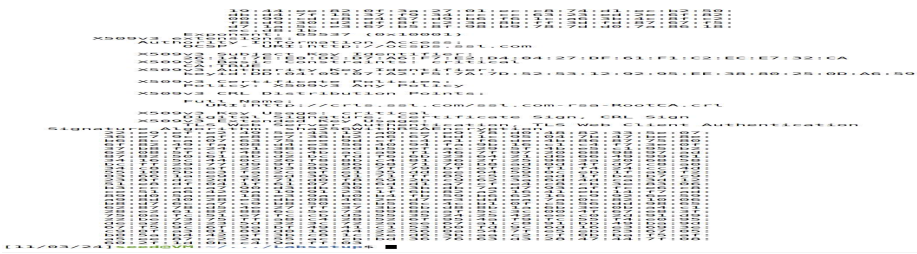
Save the intermediate certificate (certificate 1) as c1.pem:

```
-----BEGIN CERTIFICATE-----
MIIGbzCCBFegAwIBAgIICZftEJ0fB/wwDQYJKoZIhvcNAQELBQAwfDELMAkGA1UE
BhMCVVMxDjAMBgNVBAgMBVRleGFzMRAwDgYDVQQHDAdIb3VzdG9uMRgwFgYDVQQK
DA9TU0wgQ29ycG9yYXRpb24xMTAvBgNVBAMMKFNTTC5jb20gUm9vdCBDZXJ0aWZp
Y2F0aW9uIEF1dGhvcml0eSBSU0EwHhcNMTYwMjEyMTg0ODUyWhcNMzEwMjEyMTg0
ODUyWjBpMQswCQYDVQQGEwJVUzEOMAwGA1UECAwFVGV4YXMxEDAOBgNVBAcMB0hv
dXN0b24xGDAWBgNVBAoMD1NTTCBDb3Jwb3JhdGlvbjEeMBwGA1UEAwwVU1NMLmNv
bSBSU0EgU1NMIHN1YkNBMIICIjANBgkqhkiG9w0BAQEFAAOCAg8AMIICCgKCAgEA
hPYpOunhcxiF6xNzl6Tsm/Q89rnu2jVTXTBOZPaBkSD1Ic4lm7qkYwlZ/UgV5nn1
5ohhceYDC2AlR9RvGbP+26qrNcuE0XOdHJOB4SoY4d6OqLAQ6ZB0LdERK1Saa5lp
QlqHE8936dpr3hGWyqMb2LsdUuhQIzwNkLU/n9HO35irKCbKgS3Feejqkdq K5l6B
b11693o4bz9UZCUdBcQ/Xz06tA5cfnHvYkmmjxhj1lLTKwkQhWuIDrpbwWLO0QVO
c29s9ieomRKm8sYMyiBG4QqRQ/+bXwp48cF0qAByGWD6b8/gG4Xq1IBgO5p+aWFS
0mszkk5rsh4b3XbTHohP3oWQIOV20WWdtVWXiQuBB8RocAl0Ga//b+epiGgME5JX
LWXD1aDg/xHy8MUsaMlh6jDfVIFepkPnkwXDpR/n36hpgKa9dErMkgbYeEaPanLH
Yd0kv4xQ36PlMMs9WhoDErGcEG9KxAXN4Axr5wl6PTDn/lXcUFvQoIq/5CSP+Kt5
jq9tK/gRrAc4AWqRugDvQPYUm00Rqzj5Oxm5NVQYDzbyoA66CD68LETuVrfa9GuW
9MAZRO6CDzonAezIdNHsslDb1H8VN/k0zMxjI+0ub4IAmc3I5GfZtvYcpjtMj8L4
2TDS34/COov/Pf2HZ/XXGlzjZ7WPmLl4fdB6hhjs2BsCAwEAAaOCAQYwggECMDAG
CCsGAQUFBwEBBCQwIjAgBggrBgEFBQcwAYYUaHR0cDovL29jc3BzLnNzbC5jb20w
HQYDVR0OBBYEFCYUfuDc16b34tQEJ99h8cLs5zLKMA8GA1UdEwEB/wQFMAMBAf8w
HwYDVR0jBBgwFoAU3QQJB6L1en1SUxKSle44gCUNplkwEQYDVR0gBAowCDAGBgRV
HSAAMDsGA1UdHwQ0MDIwMKAuoCyGKmh0dHA6Ly9jcmxzLnNzbC5jb20vc3NsLmNv
bS1yc2EtUm9vdENBLmNybDAOBgNVHQ8BAf8EBAMCAYYwHQYDVR0lBBYwFAYIKwYB
BQUHAwEGCCsGAQUFBwMCMA0GCSqGSIb3DQEBCwUAA4ICAQAi6e/iSV5DEqDO6XjQ
SIIzXgc255yv6Oc2sqZnvRyVBHtHvo62jMoHY3Xunc/EofbeS4aHdYBvgkn6CNTj
VkCU+psWwcT3Pg83uP4k4Thu7bXvrClfS+XBlbJiCF/PSJxLrKnxRn+XIGiYl62H
glBhq9K8/fZrI2Qh1mZJmWE0FlxEDCb4i8SBNi8lmDogaFi8/yl32Z9ahmhxcLit
DU/XyKA0yOqvIrOGKH95v+/l8fQkzE1VEFvj+iyv4TXd7mRZDOsfqfIDZhrpou02
kXH/hcXlrR++t8kjj9wt8HHQ+FkryWI6bU3KPRJR6N8EH2EHi23Rp8/kyMs+gwaz
zMqnkNPbMME723rXk6/85sjOUaZCmhmRIx9rgqIWQesU962J0FruGOOasLT7WbZi
FsmSblmpjUAo49sIRi7X493qegyCEAa412ynybhQ7LVsTLEPxVbdmGVih3jVTif/
Nztr2Isaaz4LpMEo4mGCiGxec5mKr1w8AE9n6D91CvxR5/zL1VU1JCVC7sAtkdki
vnN1/6jEKFJvlUr5/FX04JXeomIjXTI8ciruZ6HIkbtJup1n9Zxvmr9JQcFTsP2c
bRbjaT7JD6MBidAWRCJWClR/5etTZwWwWrRCrzvIHC7WO6rCzwu69a+l7ofCKlWs
y702dmPTKEdEfwhgLx0LxJr/Aw==
-----END CERTIFICATE----
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
~
"c1.pem" 37L, 2293C                                                    37,25
```

Find the n value of c1.pem: openssl x509 -in c1.pem -noout -modulus

```
[11/03/24]seed@VM:~/.../Labsetup$ openssl x509 -in c1.pem -noout -modulus
Modulus=84F6293AE9E1731885EB137397A4EC9BF43CF6B9EEDA35535D304E64F6819120F521CE259BBAA4630959FD4815E679F
886171E6030B602547D46F19B3FEDBAAAB35CB84D1739D1C9381E12A18E1DE8EA8B010E990742DD1112B549A6B9969425A8713C
E9DA6BDE1196CAA31BD8BB1D52E850233C0D90B53F9FD1CEDF98AB2826CA812DC579E8EA91DA8AE65E816F5D7AF77A386F3F546
1D05C43F5F3D3AB40E5C7E71EF6249A68F1863D652D32B0910856B880EBA5BC162CED1054E736F6CF627A89912A6F2C60CCA204
0A9143FF9B5F0A78F1C174A800721960FA6FCFE01B85EAD480603B9A7E696152D26B33924E6BB21E1BDD76D31E884FDE859020E
D1659DB55597890B8107C46870097419AFFF6FE7A988680C1392572D65C3D5A0E0FF11F2F0C52C68C961EA30DF54815EA643E79
C3A51FE7DFA86980A6BD744ACC9206D878468F6A72C761DD24BF8C50DFA3E530CB3D5A1A0312B19C106F4AC405CDE00C6BE7097
30E7FE55DC505BD0A08ABFE4248FF8AB798EAF6D2BF811AC0738016A91BA00EF40F6149B4D11AB38F93B19B93554180F36F2A00
083EBC2C44EE56B7DAF46B96F4C01944EE820F3A2701ECC874D1ECB250DBD47F1537F934CCCC6323ED2E6F820099CDC8E467D9E
1CA63B4C8FC2F8D930D2DF8FC23A8BFF3DFD8767F5D71A5CE367B58F98B9787DD07A8618ECD81B
[11/03/24]seed@VM:~/.../Labsetup$
```

Find the e value from c1.pem: openssl x509 -in c1.pem -text -noout



Extract the signature from c0.pem:

```
[11/03/24]seed@VM:~/.../Labsetup$ cat signature_raw | tr -d '[:space:]:' > signature
[11/03/24]seed@VM:~/.../Labsetup$ cat signature
2b005d465f795804e57f01652d991e0a3a2b0002f7eb6901b7ae5b6e4e378d3faa3027bcc77a61d11c570dc4cff095ef84efc9f
ce5ea7ed507bec7f43088a7ae2073e0dc0b35e27e4eadcc1a319891e015fc4ef5788aea0da479b392bf97048c6846b4e272092b
c63e4f6f90da5102717f6256de49a31f641536181fe7baf37430873ed3f6892adcc1b543c8fd7320f87b2cde261df64c3845165
dfda9c429b371229d25159f4535066690e0f4323a4d900bfa57a39bf433d7f7eaa6b64dbd93729fc475f2e86e042c5d97da37bf
391c30ef7f17c38fbf9d78bed8481d58df993d0482f4b0e680722f556ee2772322df840da41cd4bdcbbb233a5531425ceea5a1c
7f347630303b31fe4640eaa7a5bf5b3cd76245d8f246f5891a38dc0f06d121f701fa8a18b02320d74e7edb586e66c8d7b149912
535e741475ae646b3b014ae44cb0e2ebfdd7073cce61e075f7c40235f425f60b7e4241ebec2528d68462e27f03875bf6fa2c703
94382887b8047d4e309b10cc232b844935c84d81e781c624cbbad68a96f889e73ac7e6b5b52ef564b57e7761c946a82497790e7
2f229a93adf8eab8cb55b04c91c0688e2c4a0872794233f6be56f8cb40a43131808cab4e94d8ad7e8267a2c145bd08f92757ce4
f4ec7b1a14b19f6f2d77a3974d4c50c71f3ba1daac097179df7725318fc06b4970c415[11/03/24]seed@VM:~/.../Labsetup$
```

Extract the body of the server certificate:
openssl asn1parse -i -in c0.pem -strparse 4 -out c0_body.bin -noout

Calculate the hash of the certificate body:

```
[11/03/24]seed@VM:~/.../Labsetup$ sha256sum c0_body.bin
a3515e747f3c25b4396481b30b02e68fce7c0d9d3cd5c33b23804188730c022d  c0_body.bin
[11/03/24]seed@VM:~/.../Labsetup$
```

Program to verify whether signature is valid or not:

This program verifies the RSA signature by decrypting it with the public key and comparing it to the expected hash of the original message. It uses the RSA formula S^e mod n, where S is the signature, e is the public exponent, and n is the modulus. If the decrypted result matches the expected hash, the signature is valid, confirming that it was created by the holder of the corresponding private key.

```c
#include <stdio.h>
#include <string.h>
#include <openssl/bn.h>
#include <openssl/sha.h>

// Function to print a BIGNUM as a hex string
void printBN(char *msg, BIGNUM *bn){
        char *hex_str = BN_bn2hex(bn);
        printf("%s %s\n", msg, hex_str);
        OPENSSL_free(hex_str);
}

int main() {
        // Initialize variables
        BIGNUM *n = BN_new(); // Modulus
        BIGNUM *e = BN_new(); // Public exponent
        BIGNUM *signature = BN_new(); // Signature to verify
        BIGNUM *expected_hash = BN_new(); // Hash of the certificate body
        BIGNUM *result = BN_new(); // Result of verification: S^e mod n
        BN_CTX *ctx = BN_CTX_new();

        // Assign values to n, e, and signature
        BN_hex2bn(&n, "84F6293AE9E1731885EB137397A4EC9BF43CF6B9EEDA35535D304E64F6819120F521CE259BBAA46
9FD4815E679F5E6886171E6030B602547D46F19B3FEDBAAAB35CB84D1739D1C9381E12A18E1DE8EA8B010E990742DD1112B549
969425A8713CF77E9DA6BDE1196CAA31BD8BB1D52E850233C0D90B53F9FD1CEDF98AB2826CA812DC579E8EA91DA8AE65E816F5
77A386F3F5464251D05C43F5F3D3AB40E5C7E71EF6249A68F1863D652D32B0910856B880EBA5BC162CED1054E736F6CF627A89
6F2C60CCA2046E10A9143FF9B5F0A78F1C174A800721960FA6FCFE01B85EAD480603B9A7E696152D26B33924E6BB21E1BDD76D
84FDE859020E576D1659DB55597890B8107C46870097419AFFF6FE7A988680C1392572D65C3D5A0E0FF11F2F0C52C68C961EA3
4815EA643E79305C3A51FE7DFA86980A6BD744ACC9206D878468F6A72C761DD24BF8C50DFA3E530CB3D5A1A0312B19C106F4AC
DE00C6BE7097A3D30E7FE55DC505BD0A08ABFE4248FF8AB798EAF6D2BF811AC0738016A91BA00EF40F6149B4D11AB38F93B19B
4180F36F2A00EBA083EBC2C44EE56B7DAF46B96F4C01944EE820F3A2701ECC874D1ECB250DBD47F1537F934CCCC6323ED2E6F8
9CDC8E467D9B6F61CA63B4C8FC2F8D930D2DF8FC23A8BFF3DFD8767F5D71A5CE367B58F98B9787DD07A8618ECD81B");
        BN_set_word(e, 65537);
        BN_hex2bn(&signature, "2b005d465f795804e57f01652d991e0a3a2b0002f7eb6901b7ae5b6e4e378d3faa3027b
a61d11c570dc4cff095ef84efc9f2fcce5ea7ed507bec7f43088a7ae2073e0dc0b35e27e4eadcc1a319891e015fc4ef5788aea
79b392bf97048c6846b4e272092b9b8c63e4f6f90da5102717f6256de49a31f641536181fe7baf37430873ed3f6892adcc1b54
d7320f87b2cde261df64c3845165e65dfda9c429b371229d25159f4535066690e0f4323a4d900bfa57a39bf433d7f7eaa6b64d
729fc475f2e86e042c5d97da37bf216391c30ef7f17c38fbf9d78bed8481d58df993d0482f4b0e680722f556ee2772322df840
cd4bdcbbb233a5531425ceea5a1c20b7f347630303b31fe4640eaa7a5bf5b3cd76245d8f246f5891a38dc0f06d121f701fa8a1
320d74e7edb586e66c8d7b14991272b535e741475ae646b3b014ae44cb0e2ebfdd7073cce61e075f7c40235f425f60b7e4241e
528d68462e27f03875bf6fa2c70320394382887b8047d4e309b10cc232b844935c84d81e781c624cbbad68a96f889e73ac7e6b
ef564b57e7761c946a82497790e791e2f229a93adf8eab8cb55b04c91c0688e2c4a0872794233f6be56f8cb40a43131808cab4
8ad7e8267a2c145bd08f92757ce4563f4ec7b1a14b19f6f2d77a3974d4c50c71f3ba1daac097179df7725318fc06b4970c415"
        BN_hex2bn(&expected_hash, "a3515e747f3c25b4396481b30b02e68fce7c0d9d3cd5c33b23804188730c022d");

        // Compute S^e mod n (this is the decrypted signature, should match the expected hash)
        BN_mod_exp(result, signature, e, n, ctx);

        BN_mask_bits(result, 256);

        // Print the decrypted signature (result) and expected hash
        printBN("Decrypted signature (S^e mod n):", result);
        printBN("Expected hash:", expected_hash);

        // Compare decrypted signature with expected hash
        if (BN_cmp(expected_hash, result) == 0){
                printf("Signature is valid.\n");
        }else{
                printf("Signature is invalid.n\n");
        }
                                                                                                    1,1
```

```
    BN_free(n);
    BN_free(e);
    BN_free(signature);
    BN_free(expected_hash);
    BN_free(result);
    BN_CTX_free(ctx);

    return 0;
                                                                                    52,1
```

```
Decrypted signature (S^e mod n): A3515E747F3C25B4396481B30B02E68FCE7C0D9D3CD5C33B23804188730C022D
Expected hash: A3515E747F3C25B4396481B30B02E68FCE7C0D9D3CD5C33B23804188730C022D
Signature is valid.
[11/03/24]seed@VM:~/.../Labsetup$
```